Tema 4

“Búsqueda Heurística Avanzada”

Año Académico 2026/27

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

Módulo II: Agentes basados en Búsqueda

Tema 2: Búsqueda no-informada

Tema 3: Búsqueda heurística

Tema 4: Búsqueda heurística avanzada

- Generación de funciones heurísticas
- Calidad de funciones heurísticas
- Más allá de A^*

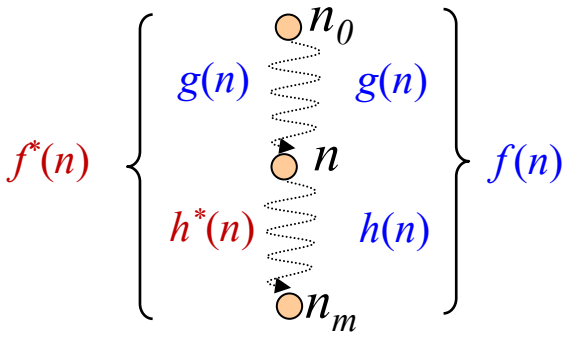
Tema 5: Búsqueda multiagente

Tema 6: Búsqueda con espacios estructurados

Tema 3: Búsqueda heurística

Algoritmo A* :

- $f^*(n) = g(n)+h^*(n)$: estimación del coste real del plan (camino) de mínimo coste que lleva del *estado inicial* a un nodo meta pasando por n
- entre las hojas del árbol de búsqueda, elegir el nodo de valor f^* mínimo
- el Algoritmo A* es *completo*
- el Algoritmo A* es *óptimo* para funciones heurísticas optimistas



Cuestiones pendientes:

- ¿ Cómo se pueden encontrar funciones heurísticas *optimistas* y/o *consistentes*?
- ¿ Cómo se puede distinguir entre "*buenas*" y "*malas*" funciones heurísticas?
- ¿Cuál es la *complejidad* del algoritmo A*? ¿Cómo se puede optimizar?

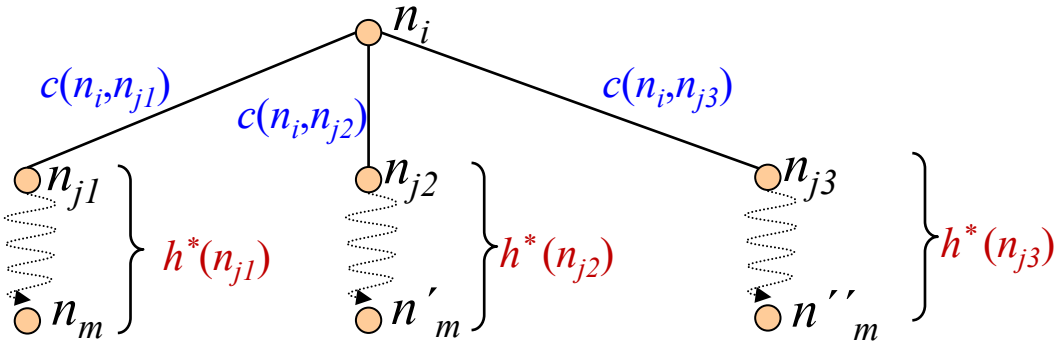
Encontrar Funciones Heurísticas: Aprendizaje

Idea: generar información heurística "sobre la marcha"

- realizar varias búsquedas (ligeramente diferentes) en el mismo dominio (p.e. siempre a Bucarest, pero desde diferentes ciudades iniciales)
- en cada paso de una búsqueda, usar el coste real de un paso parar mejorar el valor de h^*
- en la próxima búsqueda se utilizan los valores de h^* actualizados

Ejemplo:

- Suponemos que acabamos de expandir un nodo n_i con tres sucesores n_{j1}, n_{j2}, y n_{j3}



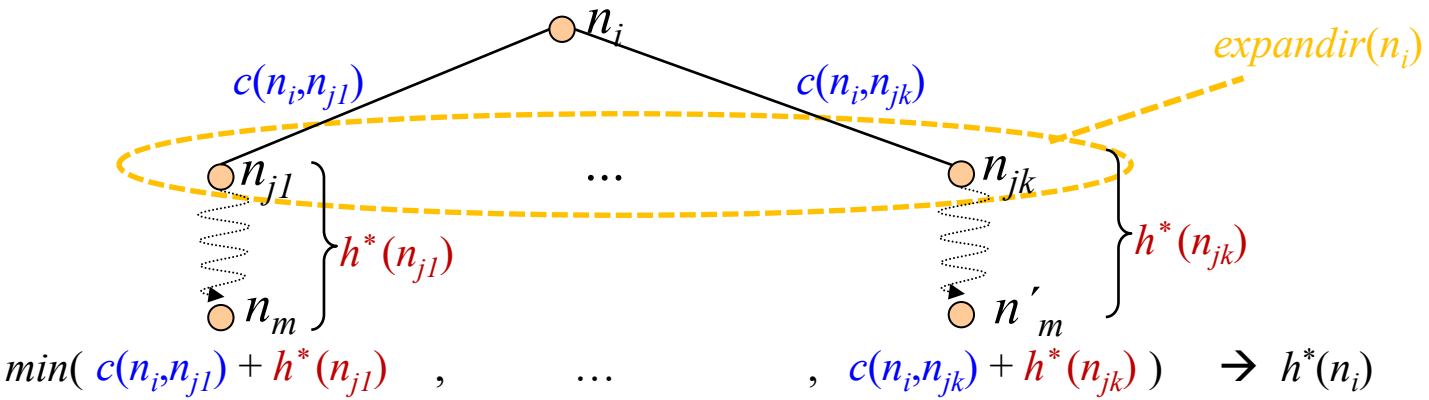
$$min(\ c(n_i,n_{j1}) + h^*(n_{j1})\ ,\ c(n_i,n_{j2}) + h^*(n_{j2})\ ,\ c(n_i,n_{j3}) + h^*(n_{j3}))\ \rightarrow\ h^*(n_i)$$

- Si $h_{j1}{}^*(n)$, $h_{j2}{}^*(n)$ y $h_{j3}{}^*(n)$ son optimistas, entonces el valor aprendido $h^*(n_i)$ es optimista

Encontrar Funciones Heurísticas: Aprendizaje

Método:

- Inicialmente, se realiza una búsqueda con $h^*(n) = 0$ para todos los nodos n
- En cada paso de n_i a n_j: $h^*(n_i) \leftarrow \min_{n_j \in expandir(n_i)} \left[h^*(n_j) + c(n_i, n_j) \right]$

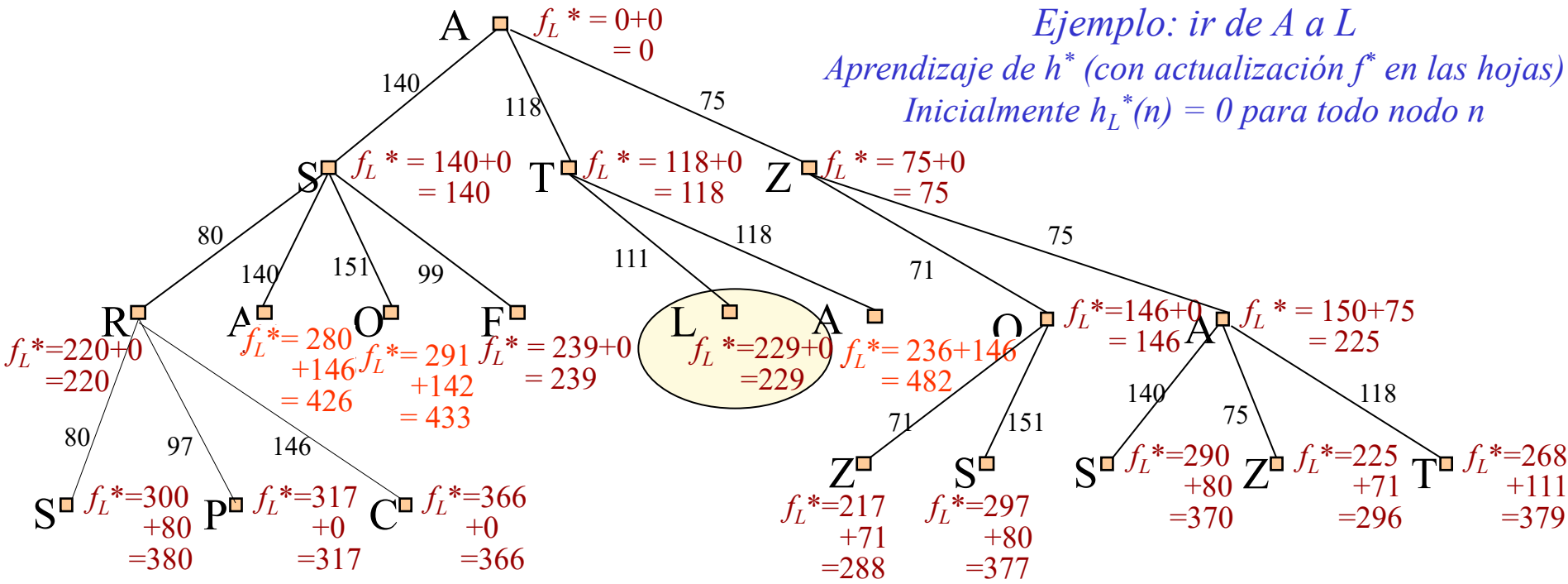


- Extensión (opcional):
  - ✓ Al aprender un nuevo valor $h^*(n)$ para un estado n, en caso de hubiera otras copias de n como hojas en el árbol de búsqueda generado por A^*, actualizar sus valores de f^*

Problema:

- Hay que almacenar los valores h^* de *todos* los nodos en una tabla (memoria!)

Ejemplo: A* con Aprendizaje de una Función Heurística



Ejemplo: ir de A a L

Aprendizaje de h^* (con actualización f^* en las hojas)

Inicialmente $h_L^*(n) = 0$ para todo nodo n

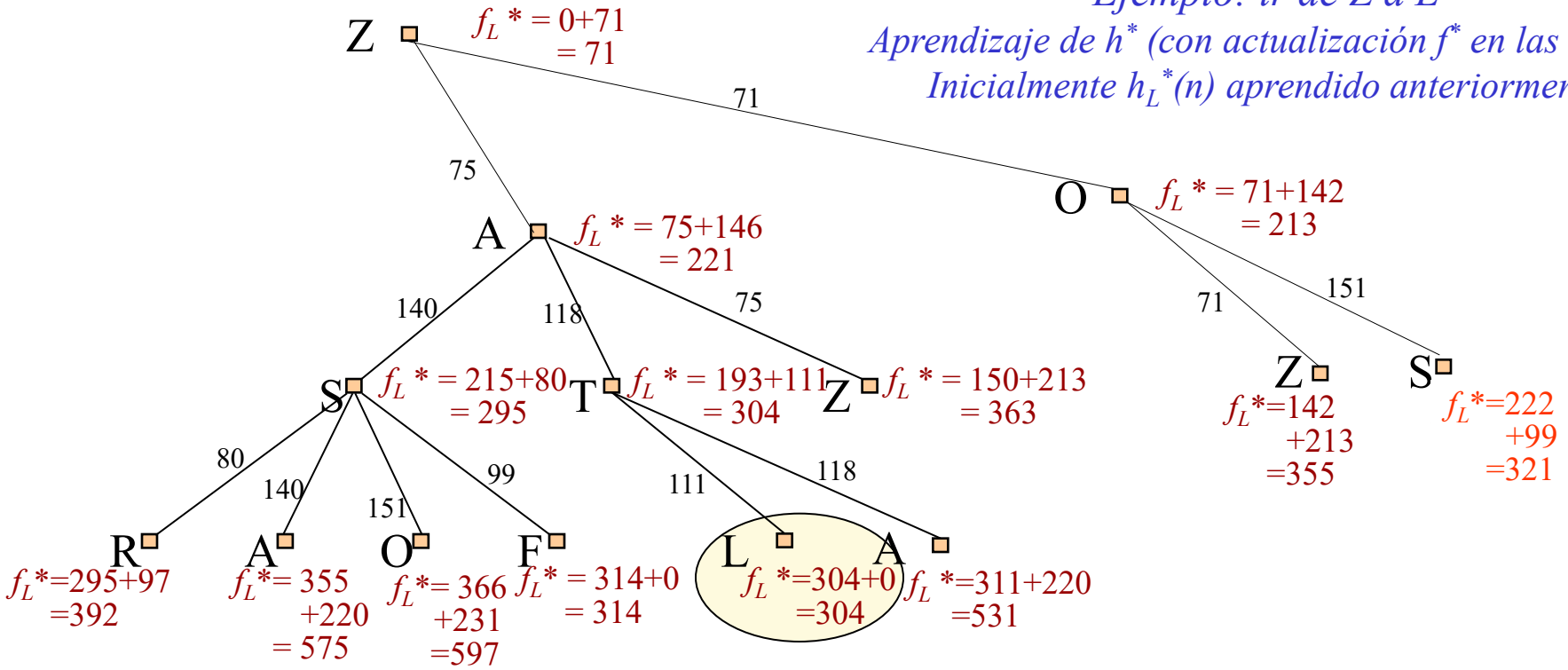
n	A	B	C	D	E	F	G	H	I	L	M	N	O	P	R	S	T	U	V	Z
h_L^*	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	~~75~~ 146												142		97	80	111			71

Ejemplo: A* con Aprendizaje de una Función Heurística

Ejemplo: ir de Z a L

Aprendizaje de h^* (con actualización f^* en las hojas)

Inicialmente $h_L^*(n)$ aprendido anteriormente



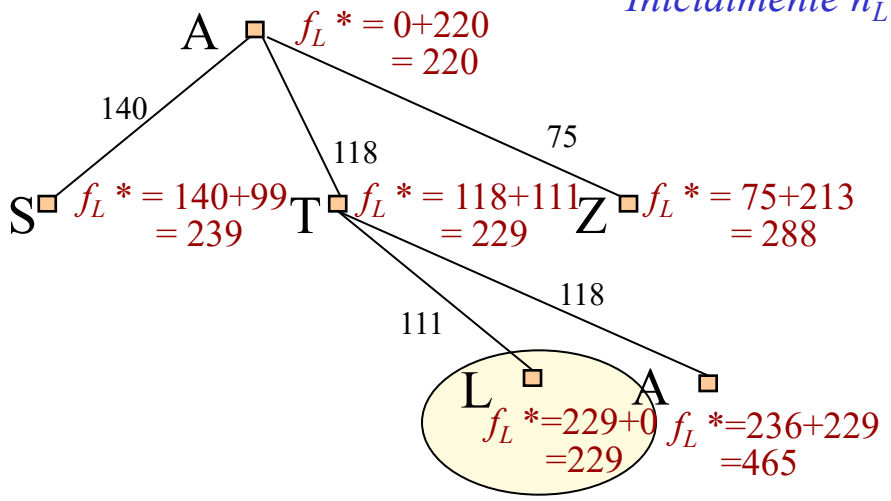
n	A	B	C	D	E	F	G	H	I	L	M	N	O	P	R	S	T	U	V	Z
h_L^*	146	0	0	0	0	0	0	0	0	0	0	0	142	0	97	80	111	0	0	71
	220												***231***			***99***	***111***			***213***

Ejemplo: A* con Aprendizaje de una Función Heurística

Ejemplo: ir de A a L

Aprendizaje de h^* (con actualización f^* en las hojas)

Inicialmente $h_L^*(n)$ aprendido anteriormente



n	A	B	C	D	E	F	G	H	I	L	M	N	O	P	R	S	T	U	V	Z
h_L^*	220	0	0	0	0	0	0	0	0	0	0	0	231	0	97	99	111	0	0	213

229 ***111***

Encontrar Funciones Heurísticas: Diseño

Opciones:

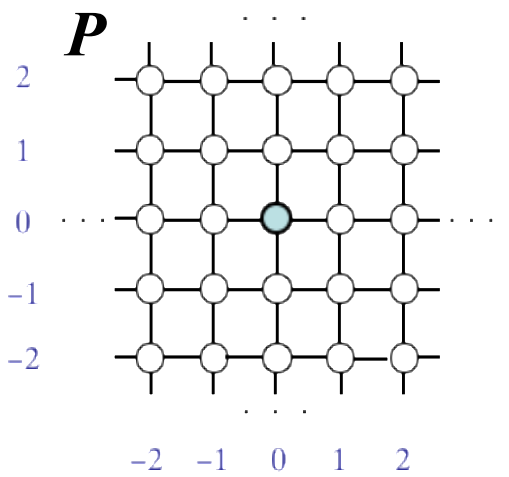
- Intuición del diseñador (p.e. similitud con heurísticas para dominios parecidos)
- Método *sistemático* de diseño (a partir de las *características* de un estado *s*)

Problemas relajados:

- Problema relajado P' con respecto a un problema original P :
  - Igual en P y en P': conjunto de estados S, estado inicial s_0, estados *meta*
  - En P' se pueden realizar las mismas acciones que en P, y algunas más (e.d., $expandir_P \subseteq expandir_{P'}$)
  - Para las acciones compartidas, el coste c es el mismo en P y en P'

Problemas relajados

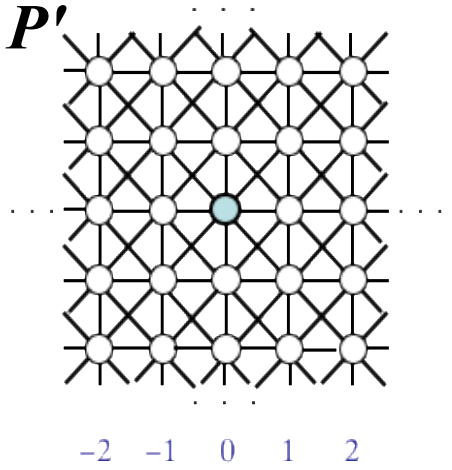
Ejemplo: Red regular 2D – estado meta: (x,y) – coste por acción: 1



$$h_P((u,v)) = |u - x| + |v - y|$$

$$h_{P'}((u,v)) = max(|u - x|, |v - y|)$$

$$h_P^*((u,v)) = max(|u - x|, |v - y|)$$



Idea:

- Por construcción, $h_{P'}(n) \leq h_P(n)$ para todo estado $s \in S$
- Por tanto, si $h_{P'}$ se puede computar fácilmente, constituye una función heurística optimista h_P^* efectiva para P

¿Cómo encontrar problemas relajados *P'* ?:

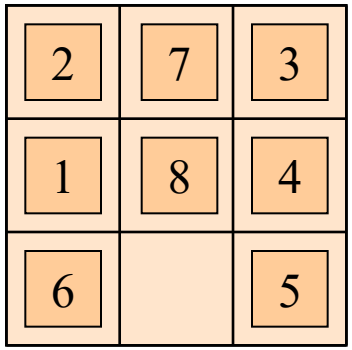
- Relajar las *precondiciones* para poder aplicar una acción a en un estado s en el problema original P

Diseñar Funciones Heurísticas

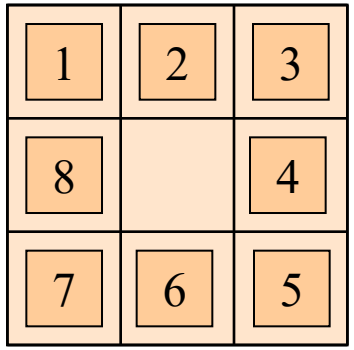
El problema del 8-puzzle:

- **Estados**:
  - posición de cada una de las piezas
- **Operadores:**
  - mover pieza adyacente a la posición del "hueco"
  - de 2 a 4 operadores aplicables, según el estado
- **Coste:**
  - La aplicación de cada operador vale *una* unidad

Estado inicial

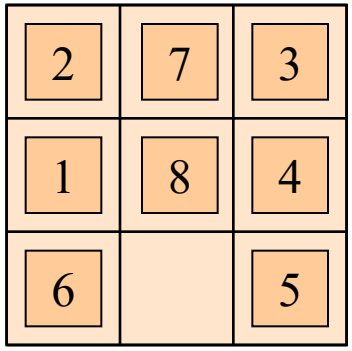


Estado meta

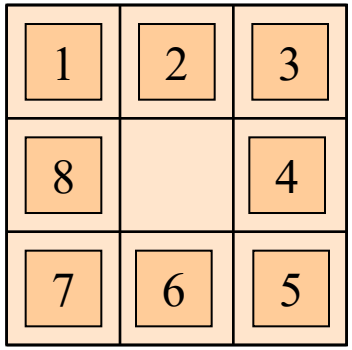


Encontrar de Funciones Heurísticas: Diseño

Estado inicial



Estado meta



Precondiciones para acciones en *P*:

- Se puede mover una ficha de X a Y en s si:
  C_1: Y está vacío en s $\wedge$ C_2: X es adyacente a Y.

Prec. para acciones en diferentes *P'*:

- Se puede mover una ficha de X a Y en s si
  - P_a': siempre (se pide ninguna de las precondiciones)
  - P_b': si Y está vacío (se pide sólo C_1)
  - P_c': si X es adyacente a Y (se pide sólo C_2)

Funciones heurísticas para *P*:

- $h_a^*(s)$ = número de *piezas descolocadas* en s
  - Ejemplo del 8-puzzle: $h_a^*(s_0) = 5$
- $h_b^*(s)$ = suma de *saltos* necesarios en s
  - Ejemplo del 8-puzzle: $h_b^*(s_0) = 5$
- $h_c^*(s)$ = suma de las *distancias de Manhattan* en s
  - Ejemplo del 8-puzzle: $h_c^*(s_0) = 1+1+1+3+1=7$

Calidad de las Funciones Heurísticas

Definición:

Sean h_1^* y h_2^* dos funciones heurísticas optimistas.

h_1^* es *más informada* que h_2^*, si para todo nodo n se cumple que

$$h_1^*(n) \geq h_2^*(n)$$

Ejemplo:

- en el 8-puzzle, h_c^* es más informada que h_a^*
  - las piezas bien colocadas no cuenta en h_a^* ni en h_c^*
  - la distancia Manhattan de cada pieza descolocada es al menos 1
  - en consecuencia, en toda posible configuración n del 8-puzzle la suma de las distancias es igual o mayor que la suma de piezas descolocadas
  - para todas las configuraciones n se cumple $h_c^*(n) \geq h_a^*(n)$

Teorema Calidad de las Funciones Heurísticas

Teorema 3: Sean h_1^* y h_2^* dos funciones heurísticas optimistas. Si h_1^* es más informada que h_2^*, entonces $A^*(h_2^*)$ expande *al menos* tantos nodos como $A^*(h_1^*)$

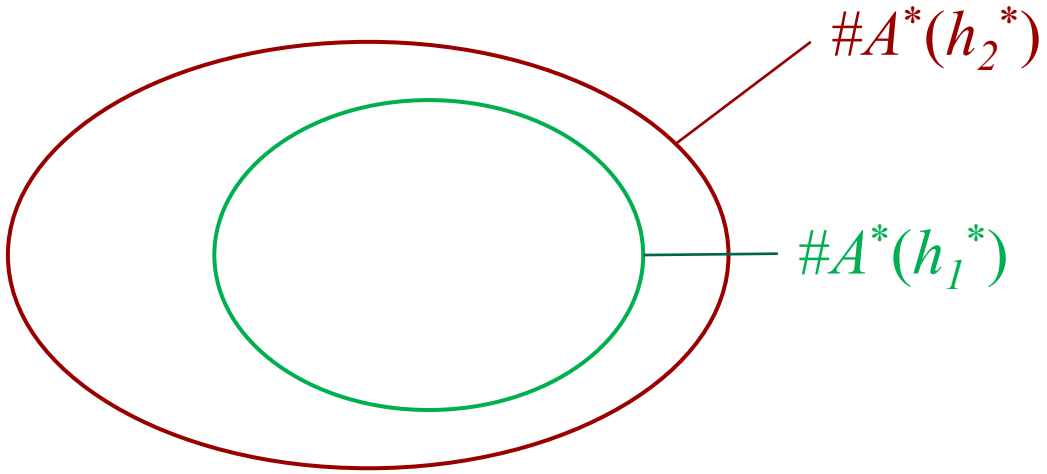
Prueba:

1. Para el mejor nodo meta n_m se cumple que $f_1^*(n_m) = f_2^*(n_m)$. Lo llamamos $f^*(n_m)$
2. Ya que h_1^* es más informada que h_2^*, para todos los nodos n se cumple que $h_1^*(n) \geq h_2^*(n)$, y por tanto $f_1^*(n) \geq f_2^*(n)$
3. Por (1) y el corolario 2 se sigue que
   - $A^*(h_1^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i). f_1^*(n_j) \leq f^*(n_m)$
   - $A^*(h_2^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i). f_2^*(n_j) \leq f^*(n_m)$
4. Por (2), se verifica que $f_1^*(n_j) \leq f^*(n_m) \rightarrow f_2^*(n_j) \leq f^*(n_m)$
5. Por (3) y (4) se concluye que cualquier nodo expandido por $A^*(h_1^*)$ también será expandido por $A^*(h_2^*)$

Teorema Calidad de las Funciones Heurísticas

Visualización de la demostración

Teorema 3: $\forall n. h_1^*(n) \geq h_2^*(n) \rightarrow \# A^*(h_1^*) \leq \# A^*(h_2^*)$



Si h_1^* es más informada que h_2^*:

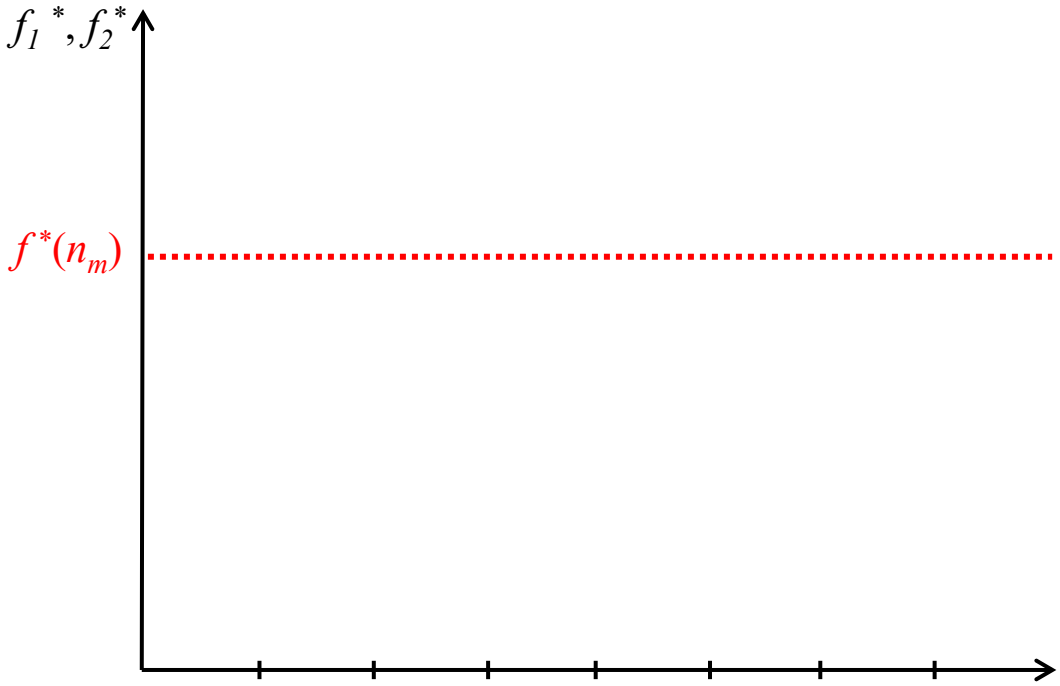
- todo nodo que es expandido por A^* con la "buena" función heurística h_1^* (la más informada) también es expandido por la "mala" función heurística h_2^*
- pero pueden existir nodos que son expandidos por la "mala" función heurística h_2^* , y **no** son expandidos por la "buena" función heurística h_1^*

Teorema Calidad de las Funciones Heurísticas

Visualización de la demostración

Teorema 3: $\forall n. h_1^*(n) \geq h_2^*(n) \rightarrow \#A^*(h_1^*) \leq \#A^*(h_2^*)$

1. Para el mejor nodo meta n_m se cumple que $f^*(n_m) = f_1^*(n_m) = f_2^*(n_m)$!

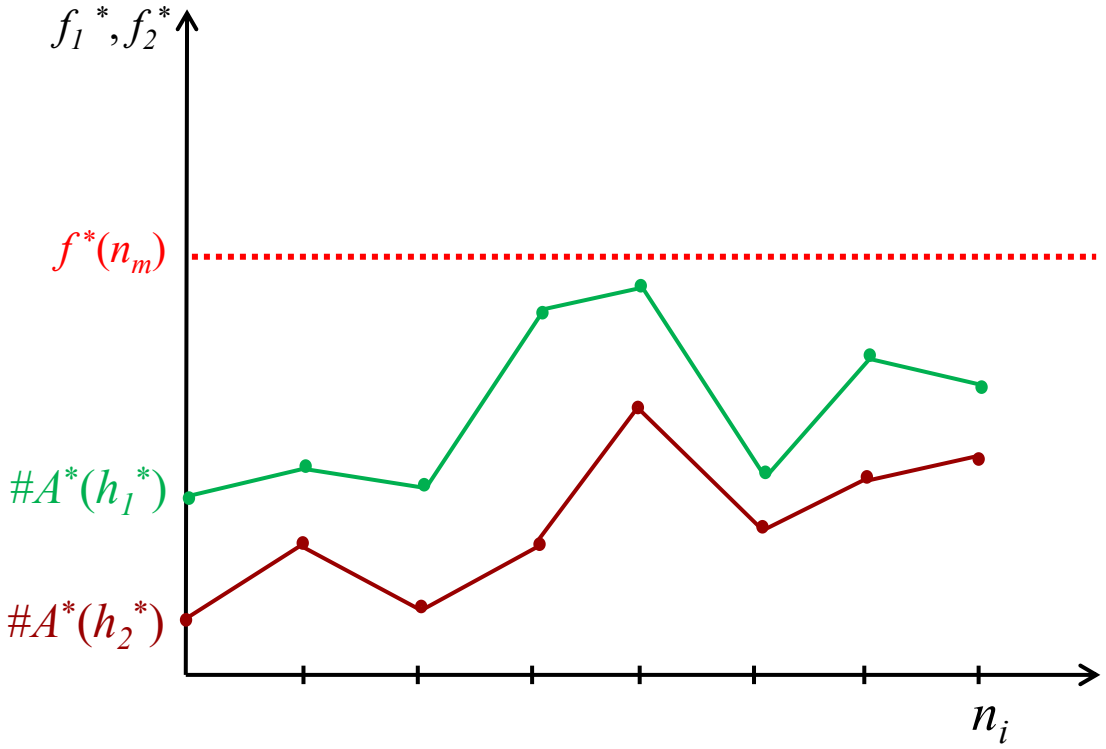


Teorema Calidad de las Funciones Heurísticas

Visualización de la demostración

Teorema 3: $\forall n.\, h_1^*(n) \geq h_2^*(n) \;\rightarrow\; \#\, A^*(h_1^*) \leq \#\, A^*(h_2^*)$

2. Ya que h_1^* es más informada que h_2^*, para todos los nodos n se cumple que $h_1^*(n) \geq h_2^*(n)$, y por tanto $f_1^*(n) \geq f_2^*(n)$

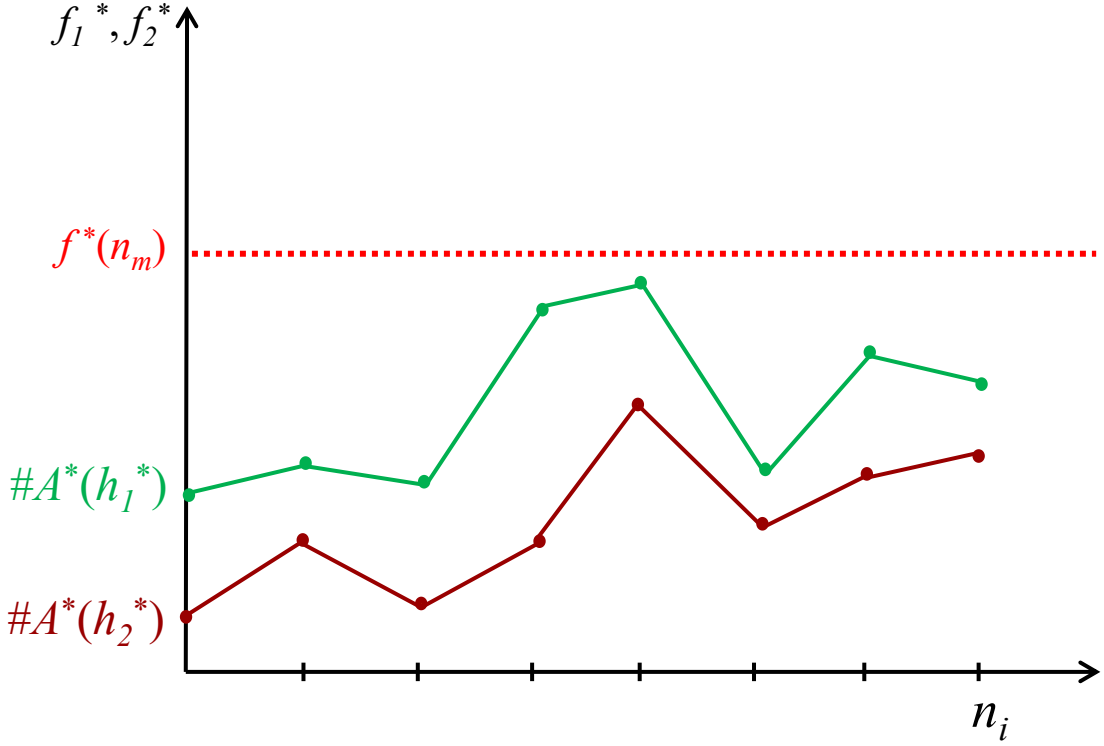


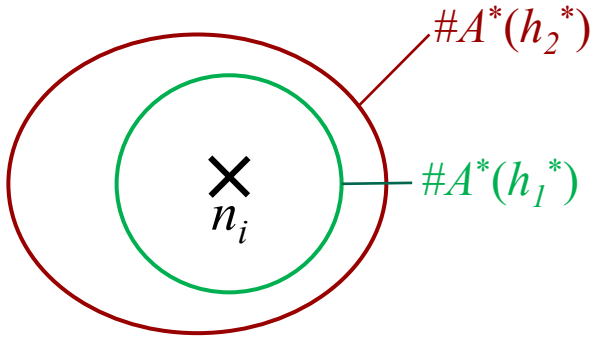
Teorema Calidad de las Funciones Heurísticas

Visualización de la demostración

Teorema 3: $\forall n.\, h_1^*(n) \geq h_2^*(n) \;\rightarrow\; \#\, A^*(h_1^*) \leq \#\, A^*(h_2^*)$

3. Por (1) y el corolario 2 se sigue que
   - $A^*(h_1^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i).\, f_1^*(n_j) \leq f^*(n_m)$
   - $A^*(h_2^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i).\, f_2^*(n_j) \leq f^*(n_m)$





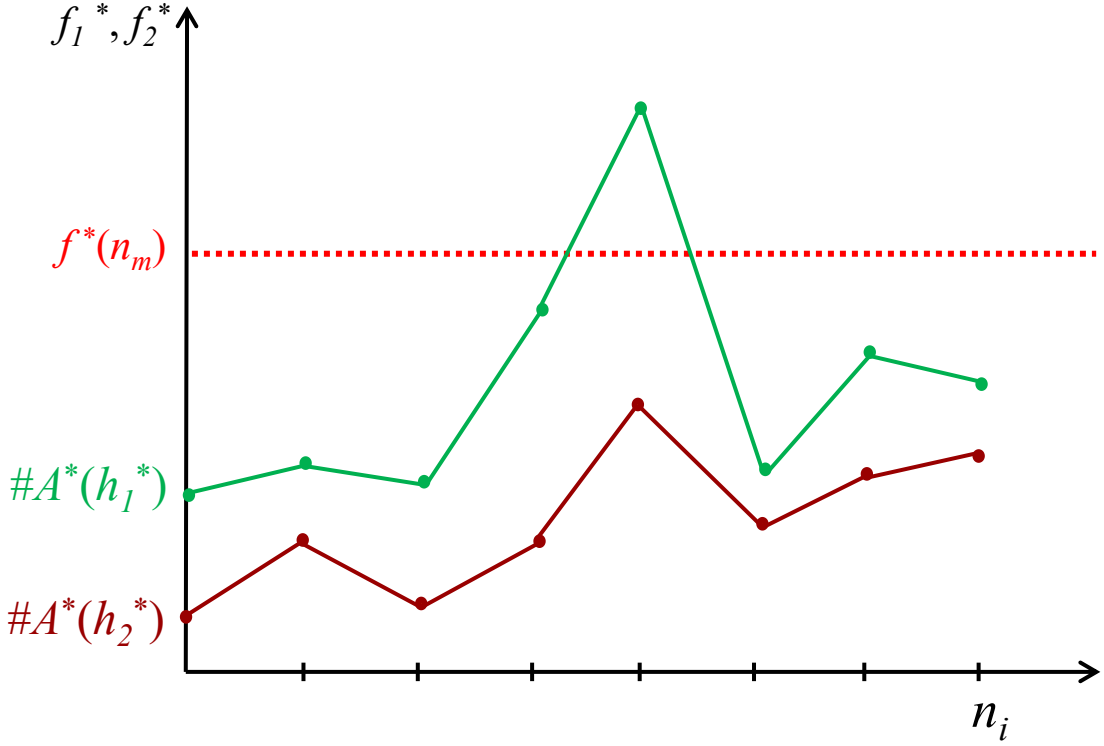
Teorema Calidad de las Funciones Heurísticas

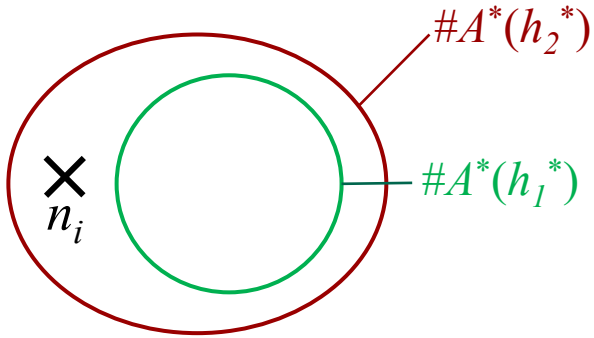
Visualización de la demostración

Teorema 3: $\forall n.\, h_1^*(n) \geq h_2^*(n) \;\rightarrow\; \#\, A^*(h_1^*) \leq \#\, A^*(h_2^*)$

Por (1) y el corolario 2 se sigue que

- $A^*(h_1^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i).\, f_1^*(n_j) \leq f^*(n_m)$
- $A^*(h_2^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i).\, f_2^*(n_j) \leq f^*(n_m)$





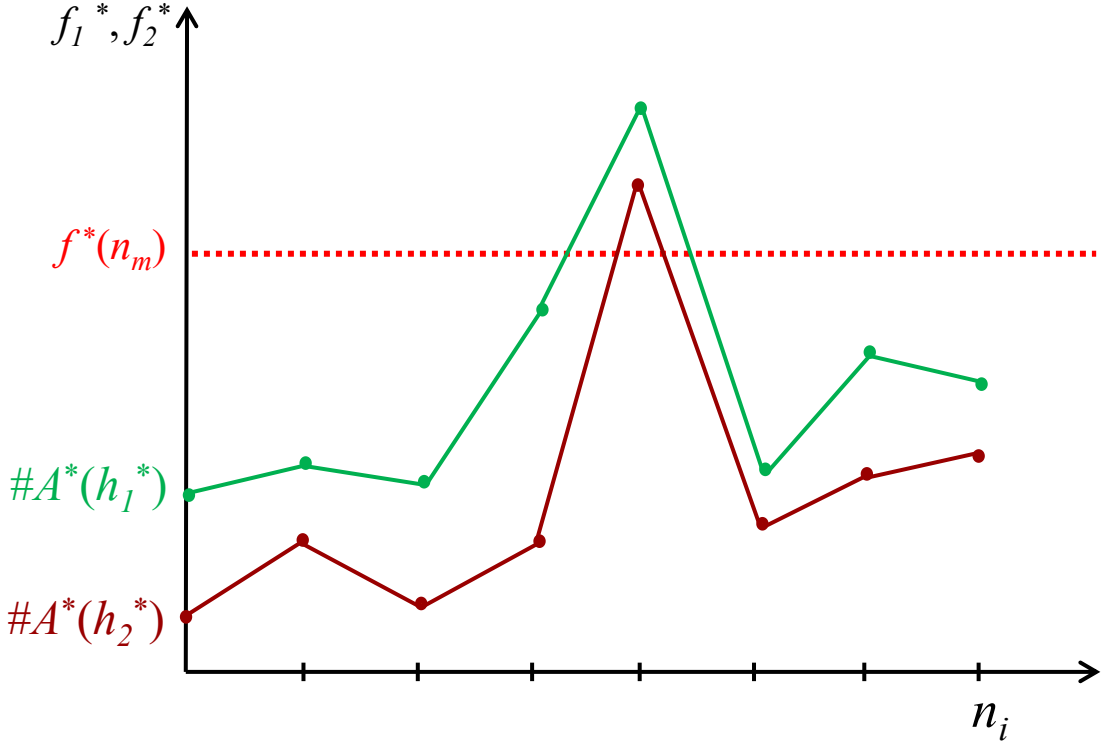
Teorema Calidad de las Funciones Heurísticas

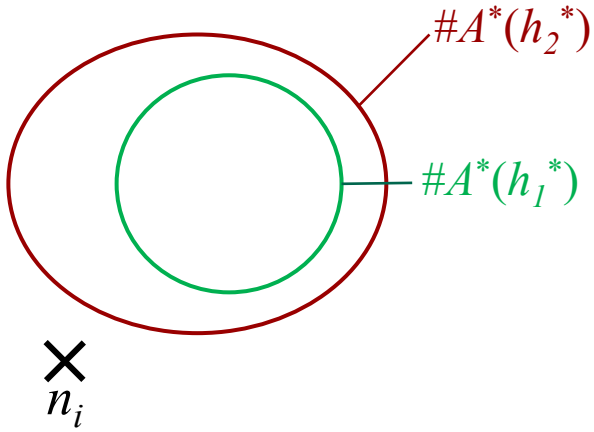
Visualización de la demostración

Teorema 3: $\forall n.\, h_1^*(n) \geq h_2^*(n) \;\rightarrow\; \#\, A^*(h_1^*) \leq \#\, A^*(h_2^*)$

Por (1) y el corolario 2 se sigue que

- $A^*(h_1^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i).\, f_1^*(n_j) \leq f^*(n_m)$
- $A^*(h_2^*)$ expande todos los nodos n_i tal que $\forall n_j \in camino_a(n_i).\, f_2^*(n_j) \leq f^*(n_m)$



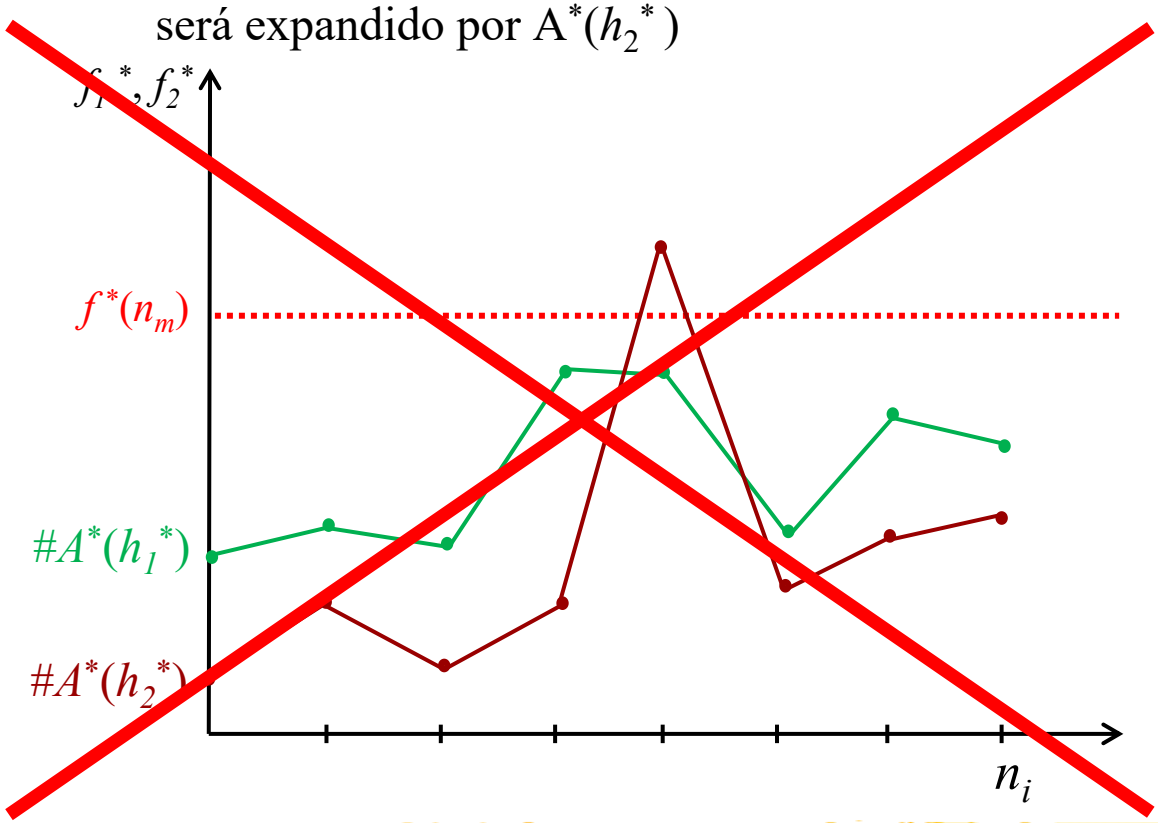


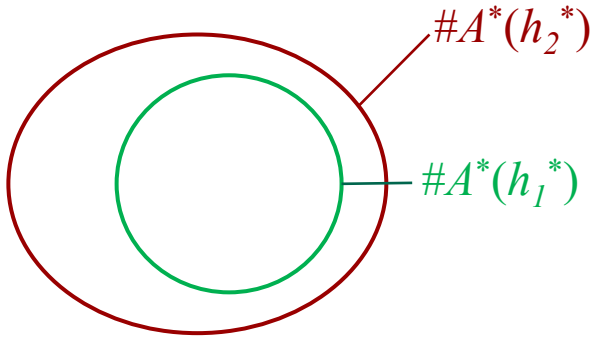
Teorema Calidad de las Funciones Heurísticas

Visualización de la demostración

Teorema 3: $\forall n.\, h_1^*(n) \geq h_2^*(n) \;\rightarrow\; \#\, A^*(h_1^*) \leq \#\, A^*(h_2^*)$

4. Por (2), se verifica que $f_1^*(n_j) \leq f^*(n_m) \rightarrow f_2^*(n_j) \leq f^*(n_m)$
5. Por (3) y (4) se concluye que cualquier nodo expandido por $A^*(h_1^*)$ también será expandido por $A^*(h_2^*)$





Calidad de las Funciones Heurísticas

Nótese:

- la calidad de la función heurística **influye** en el *número* de nodos que se expanden
- la calidad de la función heurística **no influye** en la *calidad* de la solución – con cualquier función heurística optimista A^* es óptimo (Teorema 1)

Conclusión práctica:

- preferir valores grandes de h^*, siempre que se mantenga optimista
- si hay dos funciones heurísticas optimistas h_1^* y h_2^* , y h_1^* es más informada que h_2^* :
  - ✓ se puede prescindir completamente de h_2^* en el proceso de búsqueda A^*
- si hay varias funciones heurísticas optimistas $h_1^*, \ldots, h_m^*$, sin que ninguna de ellas sea más informada que otra:
  - ✓ si todas las funciones heurísticas son "fáciles" de evaluar, se puede definir una nueva función heurística a partir de ellas

$$h^*(n) = \max\left(h_1^*(n), h_2^*(n), \ldots, h_m^*(n)\right)$$

  - ✓ por construcción, h^* es más informado que cualquier de las funciones $h_1^*, \ldots, h_m^*$

Complejidad de A*

El número de nodos expandidos por A* depende de la precisión de h^*:

- si $h^*(n) = h(n)$ para todos los nodos n:
  - información completa: complejidad lineal (sin contar la complejidad de computar h^*!)
  - calcular $h^*(n)$ suele equivaler a resolver el problema completo
- si $h^*(n) = 0$ para todos los nodos n:
  - A* degenera a la búsqueda de coste uniforme
- resultados generales:
  - en el peor caso, A* es *lineal* sólo si para todos los nodos n,
    $$| h(n) - h^*(n) | \leq O(c)$$
  - en el peor caso, A* es *polinomial* sólo si para todos los nodos n,
    $$| h(n) - h^*(n) | \leq O(\log h(n))$$
  - en escenarios reales, el *error heurístico* $| h(n) - h^*(n) |$ crece, al menos, de forma *lineal* con el coste $h(n)$ (e.d., con la distancia que separa n de un nodo meta)
  - aún así, suele haber una *mejora notable* en comparación con métodos no informados

Resultados experimentales

Comparación experimental:

- número de nodos expandidos en el problema del 8-puzzle
- varias profundidades *d* de la solución
- media sobre 100 instancias del problema

d	B. no informada (prof. iterativa)	A*(h_a)	A*(h_c)
2	10	6	6
4	112	13	12
6	680	20	18
8	6.384	39	25
10	47.127	93	39
12	3.644.035	227	73
14	—	539	113
16	—	1.301	211
18	—	3.056	363
20	—	7.276	676
22	—	18.094	1.219
24	—	39.135	1.641

Mejoras de A*: heurísticas débiles

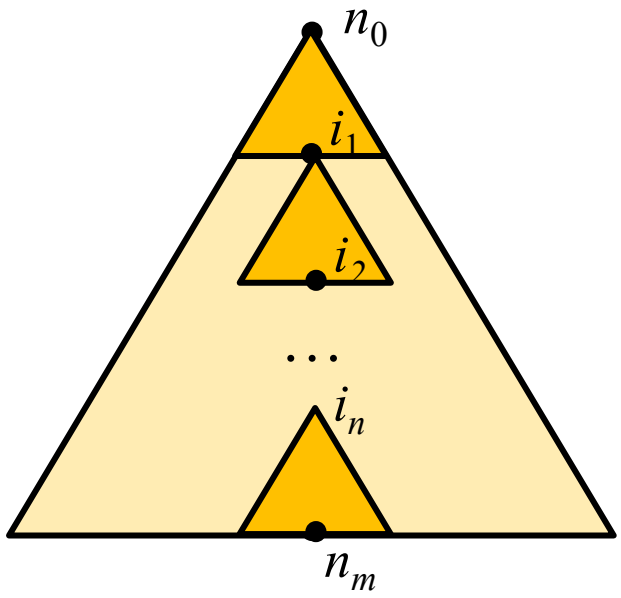
Mejoras respetando la **optimalidad**:

- atacar el problema de la complejidad en *memoria* de A^*
  (igual que la búsqueda en amplitud, A^* se queda antes sin memoria que sin tiempo)
- A costa de la expansión *repetida* de algunos nodos (aumento de tiempo de ejecución), se reduce la cantidad de nodos almacenados en memoria
  - Korf (1985): ***IDA**** (*Iterative Deepening A^**)
  - Russell (1992): ***SMA**** (*Simplified Memory-bounded A^**)
  - Korf (1992): ***RBFS*** (*Recursive Best First Search*)
- Más detalles en los libros de Russell y Norvig (sección 3.5.3 de la 3ª edición) y Nilsson (secciones 9.2.4 y 9.2.5)

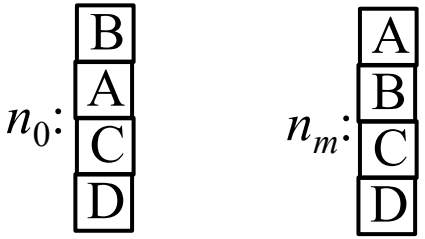
Mejoras de A*: heurísticas fuertes

Mejoras usando **heurísticas fuertes**:

- idea: *acotar* el espacio de búsqueda con información heurística
- búsqueda guiada por subobjetivos (*island-driven search*):
  - elegir estados (subobjetivos, "islas") $i_1,\ldots,i_n$ por los que se supone que una solución ha de pasar (p.e. puertos)
  - realizar búsquedas A* del estado inicial n_0 a i_1, de i_1 a i_2,…, y de i_n a la meta n_m



- La búsqueda guiada por subobjetivos **no** es óptima:
  - ejemplo en el mundo de los bloques
  - única isla i_1: todos los bloques en la mesa



Mejoras de A*: heurísticas fuertes

Búsqueda jerárquica:

- Se puede concebir como una extensión de la búsqueda por subobjetivos
- Idea: Generar los estados intermedios i_1, i_2,…, i_n de forma **dinámica**
- Método:
  - Se define un espacio de estados a un "meta-nivel" (p.e. estancia en un país)
  - macro-operadores representan acciones complejas, muchas veces ficticias (p.e. "ir de país en país")
  - realizar búsqueda A^* a meta-nivel con macro-operadores: los estados i_1, i_2,…, i_n por los que pasa el camino de la solución ("países") son los subobjetivos
  - **refinar** la solución anterior en base a la búsqueda guiada por subobjetivos
- Se pueden definir múltiples niveles de abstracción (varias búsquedas a meta niveles diferentes)

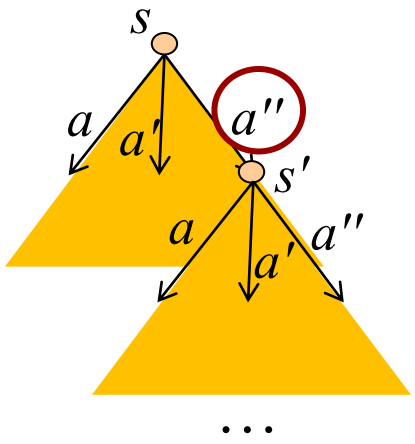
Mejoras de A*: búsqueda en línea

Todos los métodos de búsqueda vistos hasta ahora son *offline*:

- Primero se genera un plan (secuencia de acciones) que, en el modelo (espacio de estados), lleve del estado actual a un estado meta
- Después se ejecutan las acciones de este plan en el mundo real

Búsqueda en línea (*online search*):

- *clase* de métodos que "engranan" búsqueda y acción/percepción en un *ciclo*:
  1. Se percibe el estado actual del mundo *s*
  2. Se aplica un método de búsqueda online para decidir la *siguiente* acción *a*
  3. Se ejecuta la acción *a* en el mundo real, dando lugar al nuevo estado *s'*



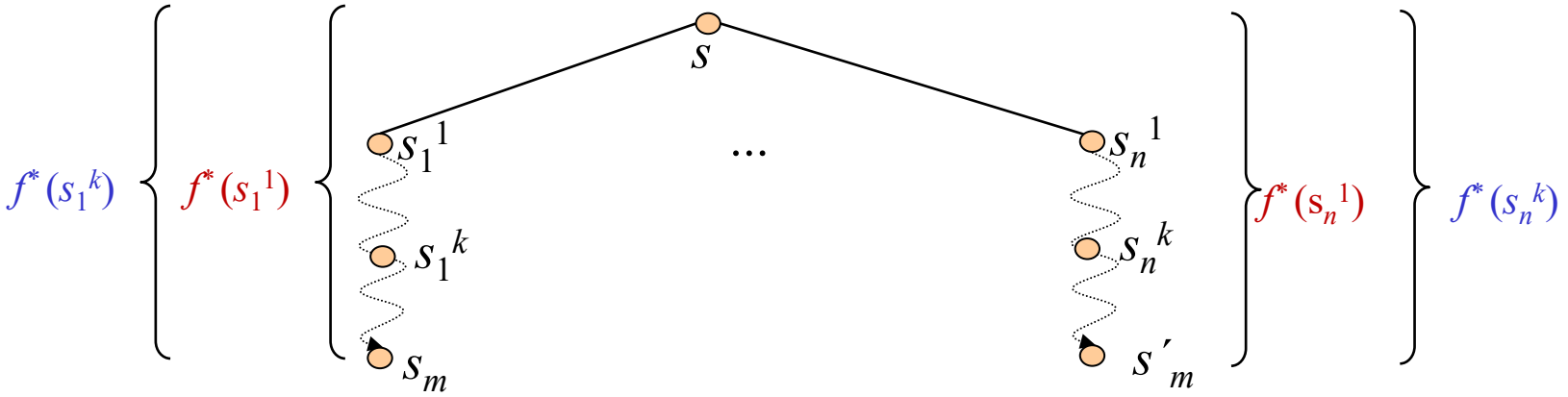
Mejoras de A*: búsqueda en línea

Búsqueda en línea (*online search*):

- indicado cuando:
  - el espacio de búsqueda es demasiado grande para buscar hasta la solución (y no se pueden aplicar las técnicas anteriores)
  - no es realista usar un modelo determinista de los efectos de acciones
    - por frecuentes contingencias (p.e.: a veces el brazo deja caer un bloque)
    - porque no se dispone de un modelo del entorno (p.e.: un "mapa")
- medida de eficiencia:
  - en general, no se puede asegurar optimalidad ni completitud
  - minimizar el *índice competitivo* $= \frac{\textit{Coste del camino real del agente}}{\textit{Coste del camino óptimo}}$
  - el índice competitivo puede ser infinito, particularmente cuando hay acciones que no son reversibles

Búsqueda con horizonte

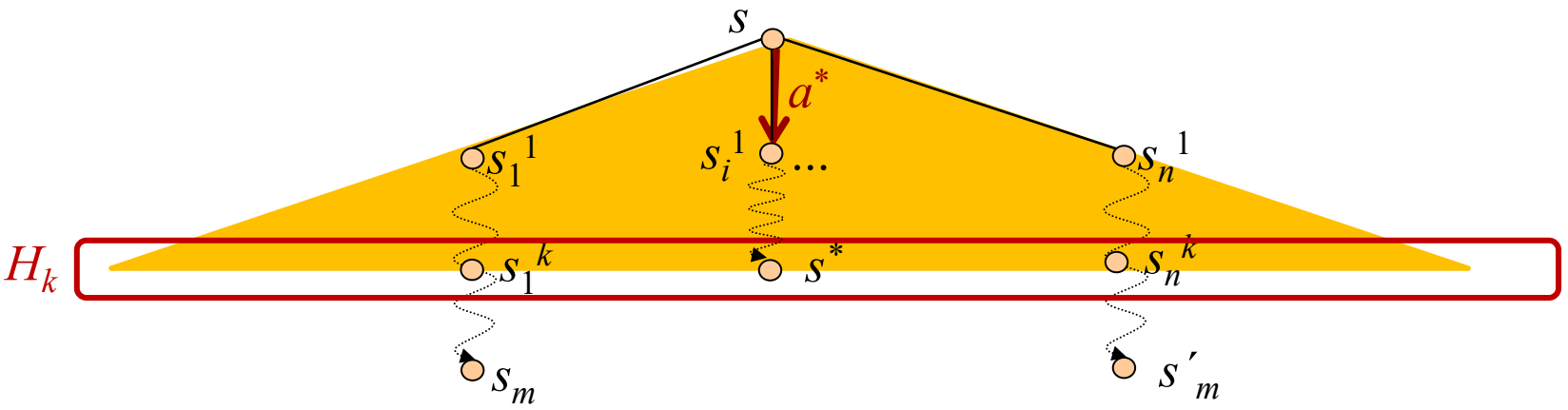
- Problema de decisión de un agente en una búsqueda en línea:
  - ✓ Estando en un estado actual s, y habiendo n acciones posibles en s,
  - ✓ ¿a cual de los estados sucesores s_1^1, .., s_n^1 ir? ("físicamente", e.d., en el mundo real)



- **Búsqueda con horizonte** (*limited-horizon search*): idea
  - ✓ Los valores de f^* de los estados s_i^1 estiman el coste total de un camino desde el estado actual hasta un estado meta que pasa por s_i^1
  - ✓ Además, a mayor profundidad en el árbol de búsqueda, menor el error heurístico de h^*
  - ✓ Por tanto, en general los valores de f^* de nodos "más profundos" s_i^k en los caminos que se inician en s_i^1 estiman mejor el coste total de estos caminos que $f^*(s_i^1)$

Búsqueda con horizonte: Método

- Método: buscar k acciones hacia adelante, luego decidir la siguiente acción y ejecutarla
  - Generar el conjunto H_k de estados a los que se puede llegar con k acciones.
  - *Heurística (fuerte)* que aplica la búsqueda con horizonte:
    - el mejor camino desde s a un estado meta es a través del estado s^* a nivel k de menor valor de f^* (i.e. $n^* \leftarrow \arg\min_{n \in H_k} [f^*(n)]$)
  - Por tanto, se realiza la acción a^* que lleva al primer estado s_i^1 en este camino
  - Luego, estando "físicamente" en s_i^1, se repite el proceso

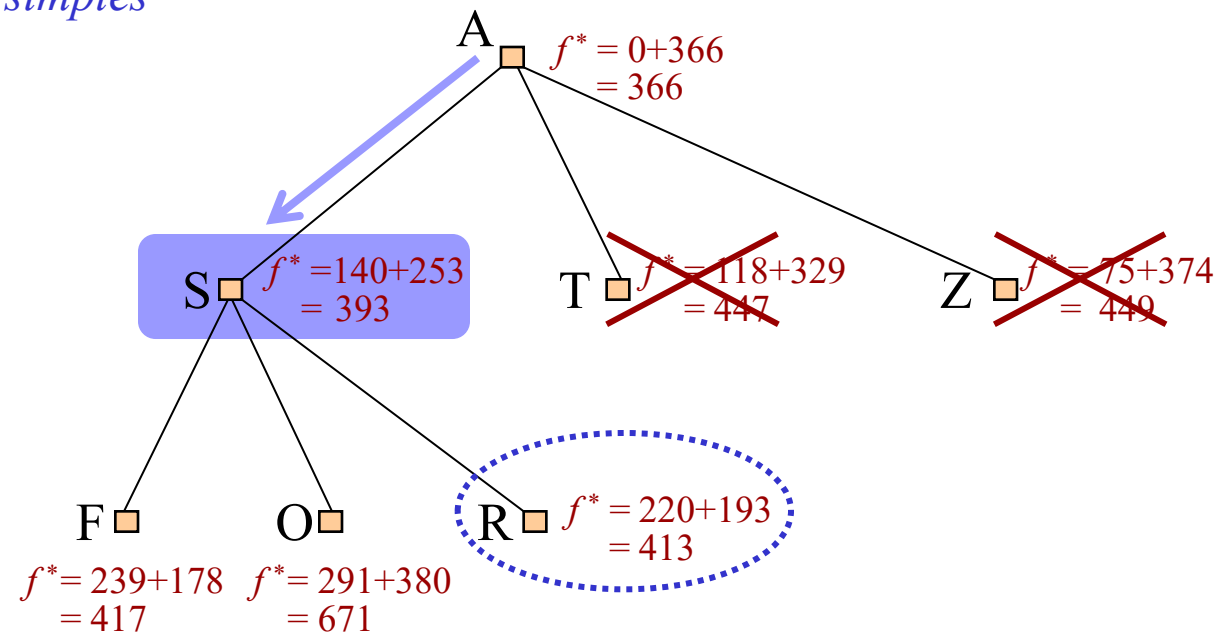


Búsqueda con horizonte: Algoritmo

- Repetir hasta que el agente se encuentre en un estado meta:
  - Percibir el estado actual s
  - Aplicar *búsqueda en profundidad limitada* por el nivel k
  - En cada nodo hoja n de nivel k, actualizar un valor α que representa el mínimo valor de f^* encontrado en nodos de nivel k hasta el momento, e.d.:
    - Inicializar $\alpha \leftarrow +\infty$
    - En el nodo n (de nivel k) realizar: $\alpha \leftarrow \min(\alpha, f^*(n))$
  - Si en la búsqueda en profundidad limitada se ha encontrado un nodo meta, sea n^* este nodo. De lo contrario, sea n^* el primer nodo hoja (de nivel k) con $f^*(n^*) = \alpha$
  - Ejecutar la primera acción a^* en el camino que lleva de s a n^*
- Optimización mediante *podas* α (para h^* consistente)
  - Si h^* es consistente, para cualquier nodo interior n' del árbol de búsqueda (e.d., de nivel $k' < k$), los sucesores de n' tendrán valor de f^* mayor o igual a $f^*(n')$ [Lema 1]
  - Por tanto, durante el proceso de búsqueda en profundidad limitada, se puede abandonar cualquier rama a partir de un nodo n' con $f^*(n') \geq \alpha$

Ejemplo: Búsqueda con horizonte

estado actual s = A
horizonte k = 2
filtrando ciclos simples

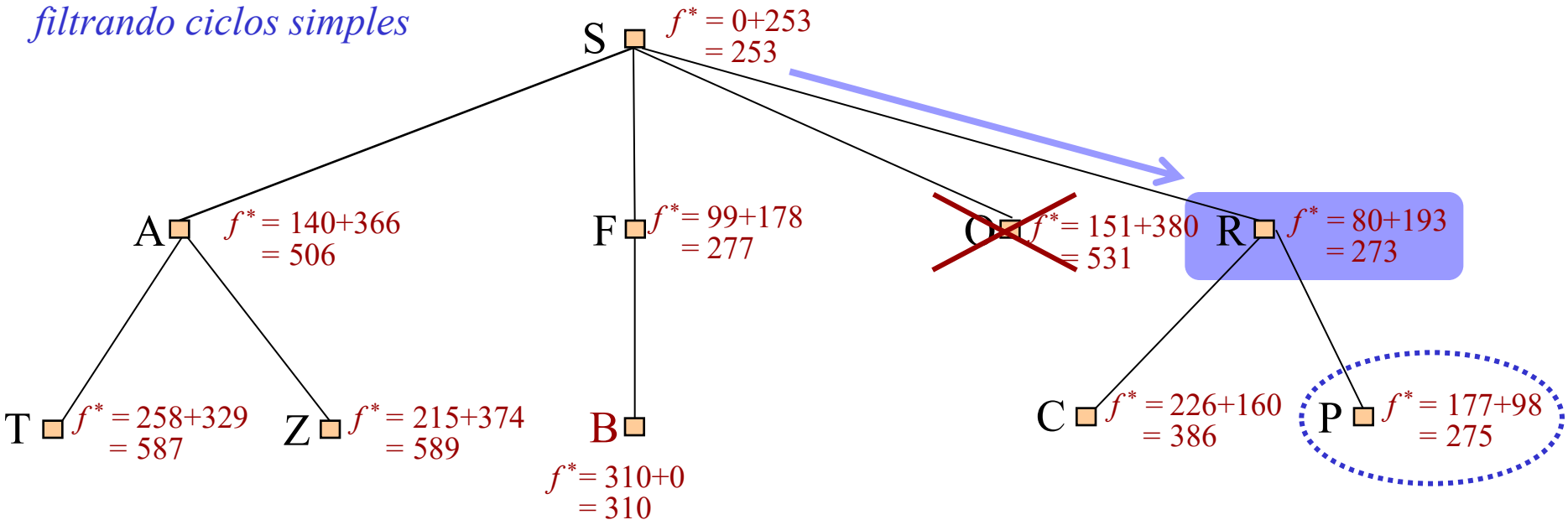


k=2

$\alpha = 417$ $\alpha = 417$ $\alpha = 413$

Ejemplo: Búsqueda con horizonte

estado actual s = S
horizonte k = 2
filtrando ciclos simples

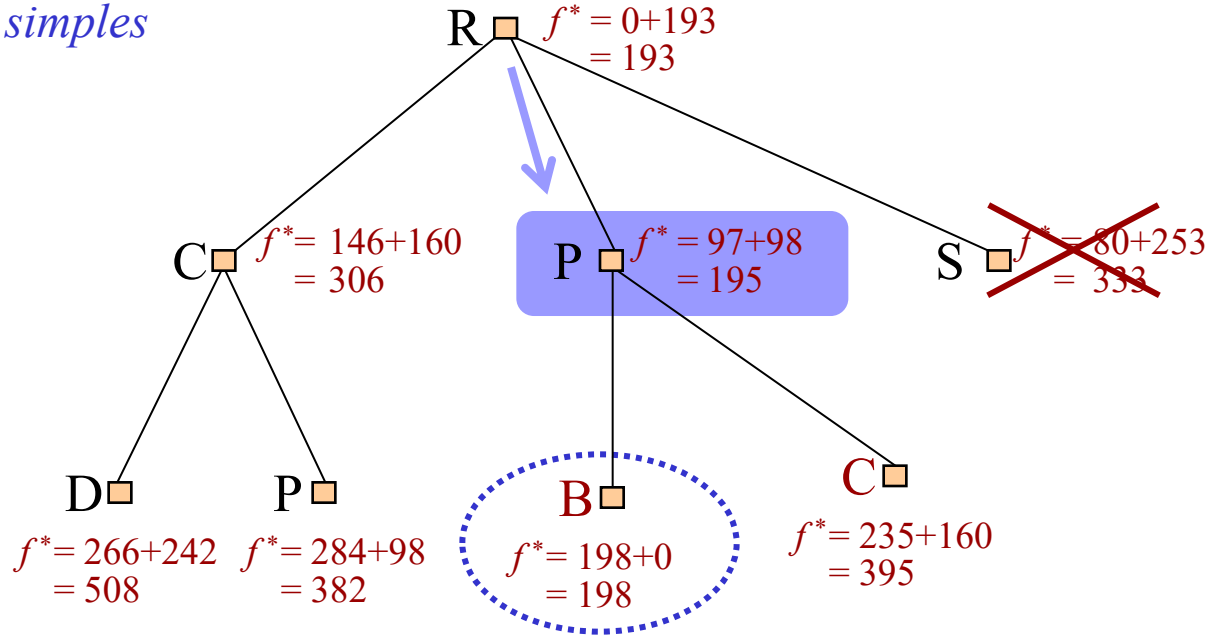


k=2

$\alpha = 587$ $\alpha = 587$ $\alpha = 310$ $\alpha = 310$ $\alpha = 275$

Ejemplo: Búsqueda con horizonte

estado actual s = R
horizonte k = 2
filtrando ciclos simples

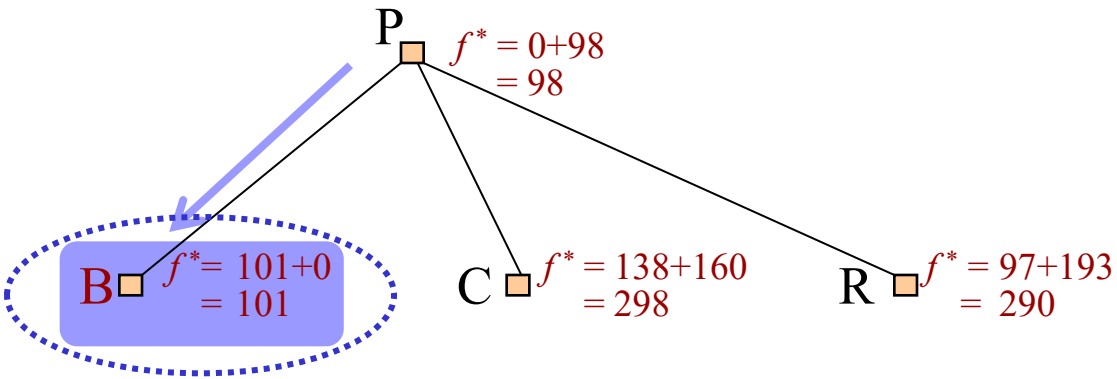


k=2

$\alpha = 508$ $\alpha = 382$ $\alpha = 198$ $\alpha = 198$

Ejemplo: Búsqueda con horizonte

estado actual s = P
horizonte k = 2
filtrando ciclos simples



k=2